

```

#define LDR1_PIN 34      // Top Left
#define LDR2_PIN 35      // Top Right
#define LDR3_PIN 32      // Bottom Left
#define LDR4_PIN 33      // Bottom Right

// Horizontal Motor
const int motorAzimuthPins[4] = {19, 18, 5, 17};

// Vertical Motor
const int motorElevationPins[4] = {16, 4, 2, 15};

// Half-step sequence for 28BYJ-48
const byte stepSequence[8][4] =
{
    {1,0,0,0},
    {1,1,0,0},
    {0,1,0,0},
    {0,1,1,0},
    {0,0,1,0},
    {0,0,1,1},
    {0,0,0,1},
    {1,0,0,1}
};

const int deadZone = 200;
const int stepDelay = 3;
const int maxSteps = 4;

bool horizontalInvert = false;
bool verticalInvert = false;

int stepIndexH = 0;
int stepIndexV = 0;

void setup()
{
    Serial.begin(115200);

    analogReadResolution(12);

    for(int i=0;i<4;i++)
    {

```

```

        pinMode(motorAzimuthPins[i],OUTPUT);
        pinMode(motorElevationPins[i],OUTPUT);

        digitalWrite(motorAzimuthPins[i],LOW);
        digitalWrite(motorElevationPins[i],LOW);
    }

    Serial.println("Dual Axis Solar Tracker");
}

void loop()
{
    int topLeft = analogRead(LDR1_PIN);
    int topRight = analogRead(LDR2_PIN);
    int bottomLeft = analogRead(LDR3_PIN);
    int bottomRight = analogRead(LDR4_PIN);

    int left = topLeft + bottomLeft;
    int right = topRight + bottomRight;

    int top = topLeft + topRight;
    int bottom = bottomLeft + bottomRight;

    int horizontalError = left - right;
    int verticalError = top - bottom;

    moveAxis(horizontalError,
              motorAzimuthPins,
              horizontalInvert,
              stepIndexH);

    moveAxis(verticalError,
              motorElevationPins,
              verticalInvert,
              stepIndexV);

    Serial.print("TL=");
    Serial.print(topLeft);

    Serial.print(" TR=");
    Serial.print(topRight);

```

```

Serial.print(" BL=");
Serial.print(bottomLeft);

Serial.print(" BR=");
Serial.print(bottomRight);

    delay(150);
}

void moveAxis(int error,
              const int pins[4],
              bool invert,
              int &stepIndex)
{
    if(abs(error)<deadZone)
        return;

    int direction;

    if(error>0)
        direction=1;
    else
        direction=-1;

    if(invert)
        direction=-direction;

    int steps=min(maxSteps,abs(error)/500+1);

    for(int i=0;i<steps;i++)
        stepMotor(pins,direction,stepIndex);
}

void stepMotor(const int pins[4],
               int direction,
               int &stepIndex)
{
    stepIndex=(stepIndex+direction+8)%8;

    for(int i=0;i<4;i++)
        digitalWrite(pins[i],stepSequence[stepIndex][i]);
}

```

```
    delay(stepDelay);  
}
```